

The background of the slide is a high-quality aerial photograph of a Swiss landscape. In the foreground, a vibrant blue lake is nestled between lush green, forested mountains. The middle ground shows rolling green hills with small clusters of buildings and winding roads. In the far distance, more mountain ranges are visible under a vast sky filled with soft, white clouds. The overall color palette is dominated by blues, greens, and whites, creating a serene and majestic atmosphere.

GitOps Needs an API

Swiss Cloud Native Day 2026 · Simon Koudijs

Simon Koudijs

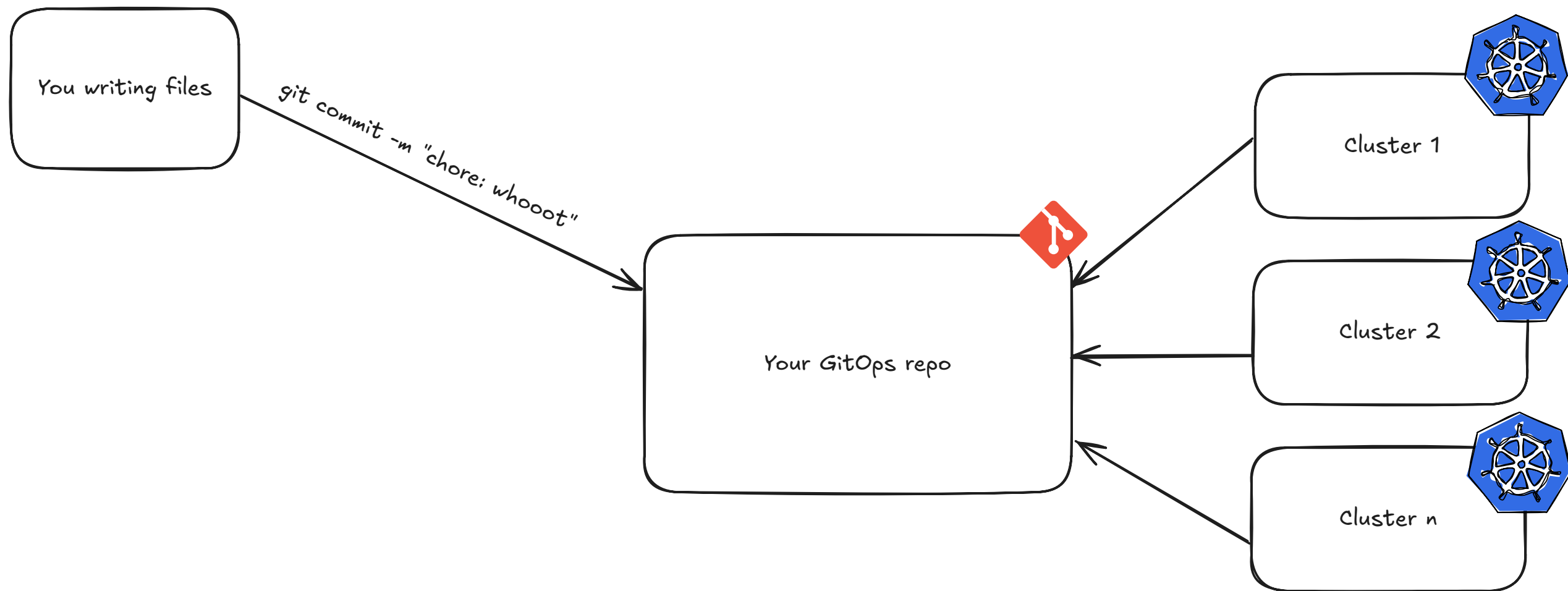
[Software|Product|Cloud|AI] engineer

- From NL
- Left my job 14 months ago to build my own company
- koudijs.dev: consultancy, training, speaking
- reversegitops.dev: the pattern, and a manifest
- [ConfigButler](#): hosted product of this idea



GitOps is great

- Desired state
- History
- Rollback
- Great tools like ArgoCD and FluxCD



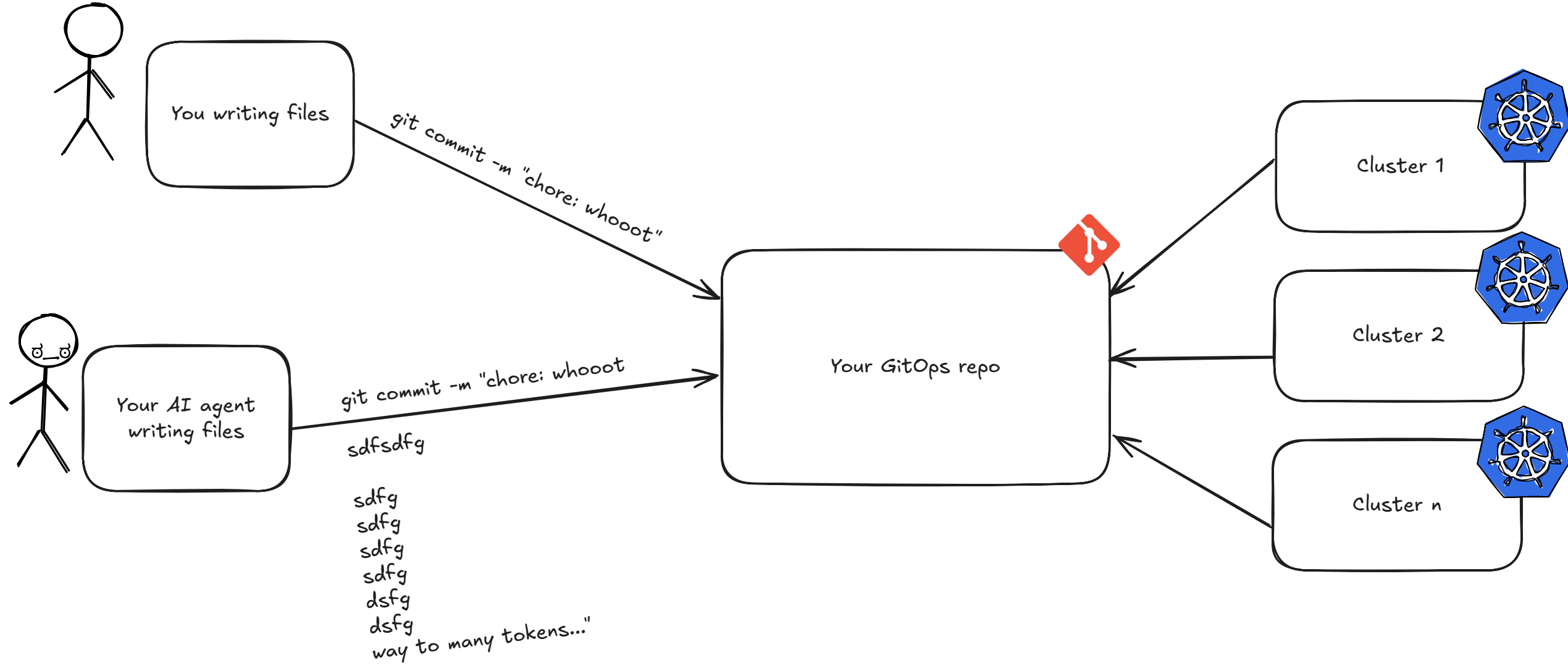
Every commit answers

Who 

When 

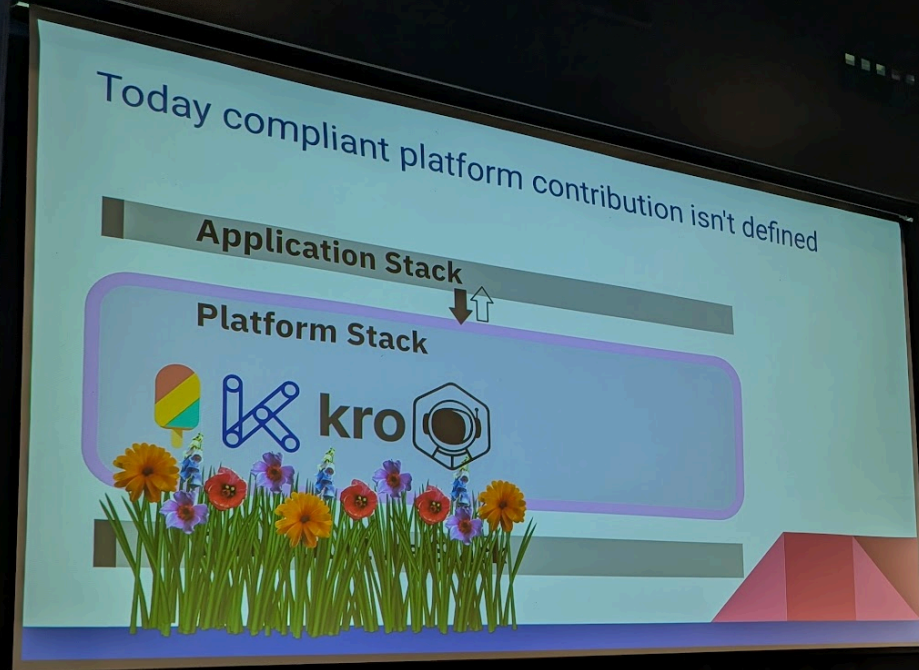
What 

Why  (intent, message)

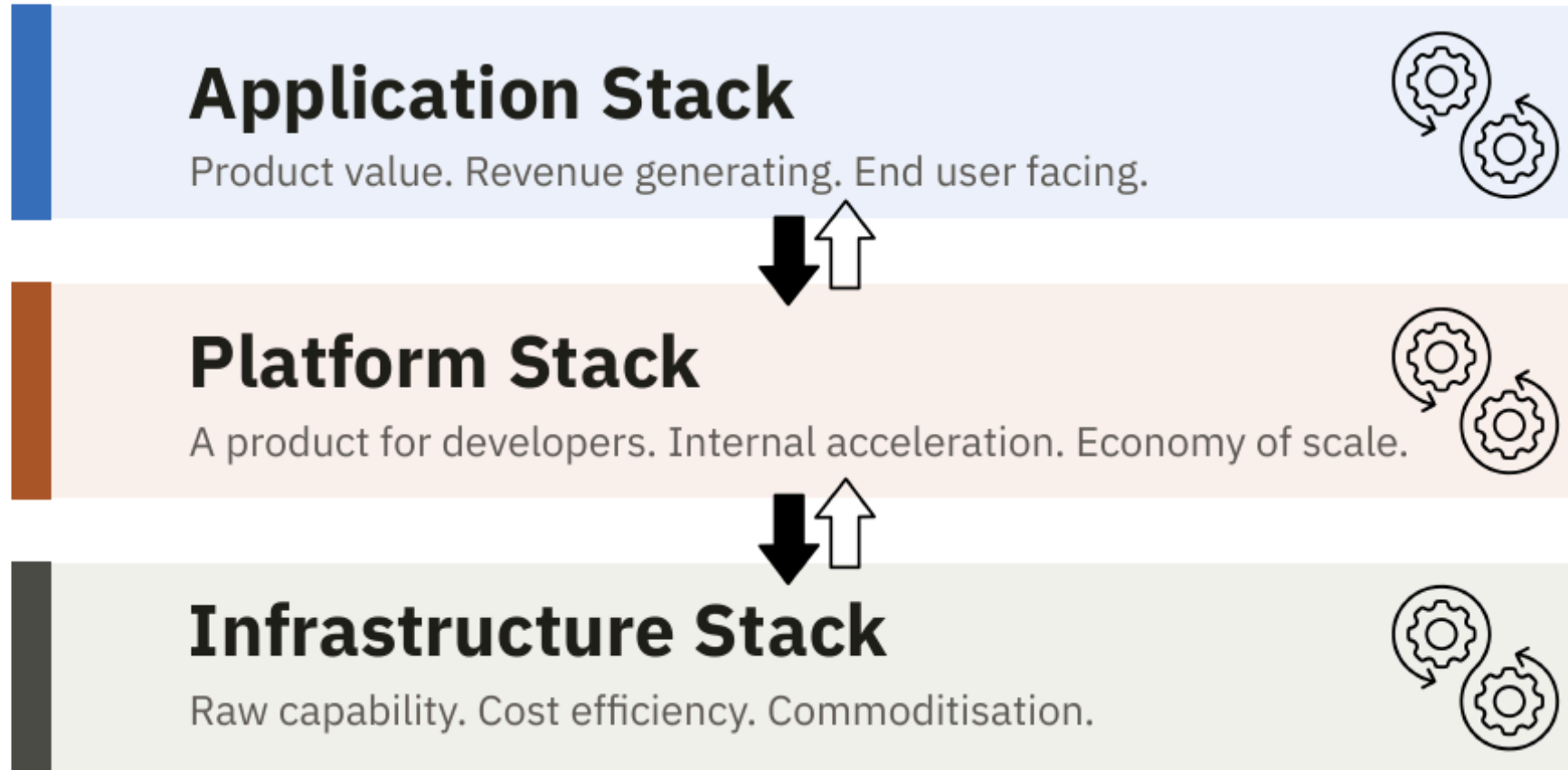


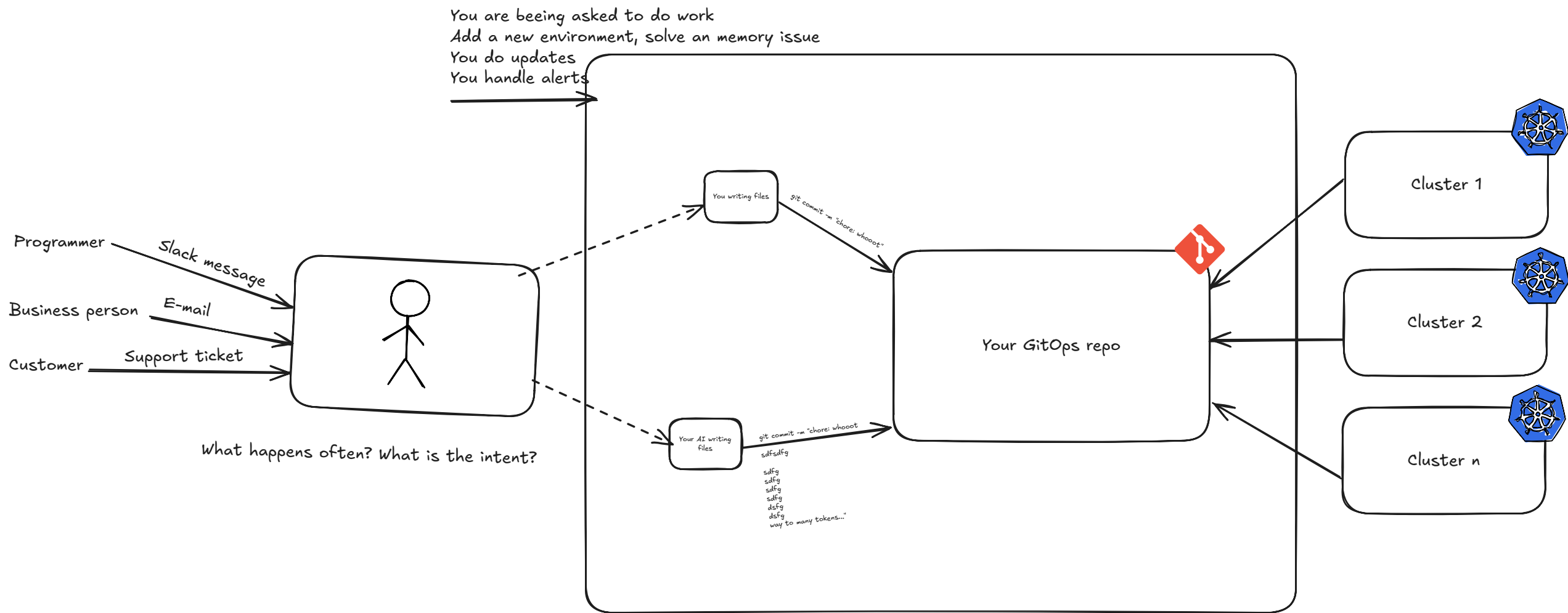
Business intent

- Which apps does my specific customer want to run
- Network ranges for my cluster
- Self-servicing clusters
- Etc.



Within stacks YBIYRI, between stacks use APIs





But GitOps also has problems

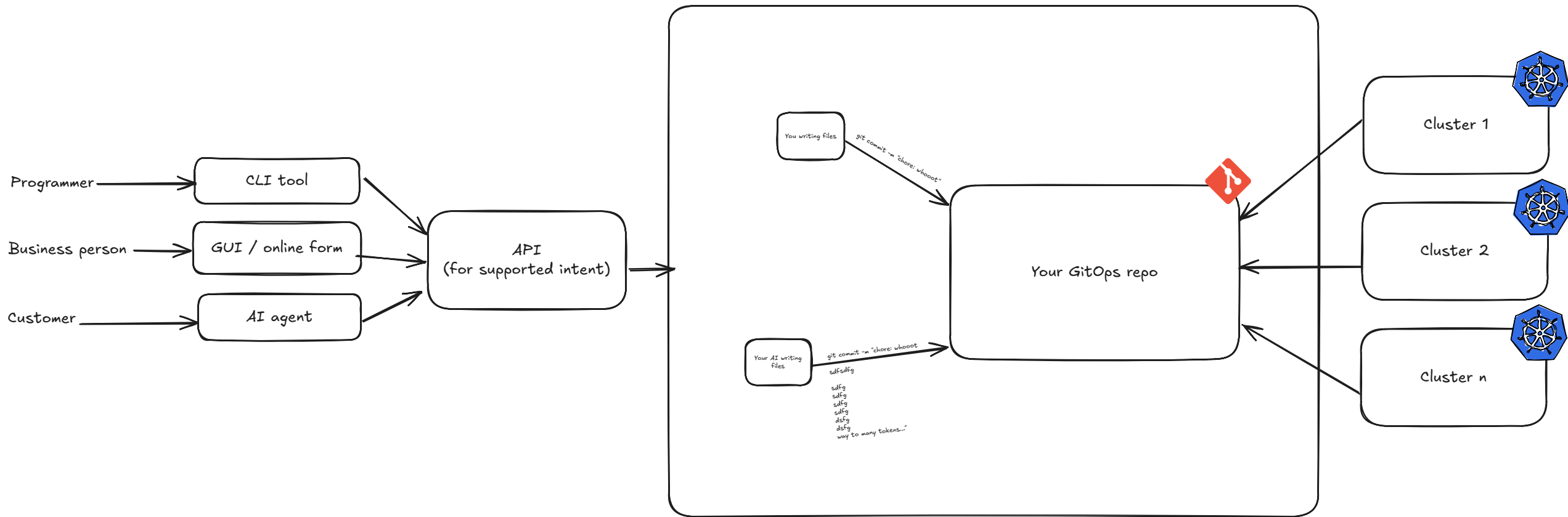
Complex constructs are possible, and if you give people the option...

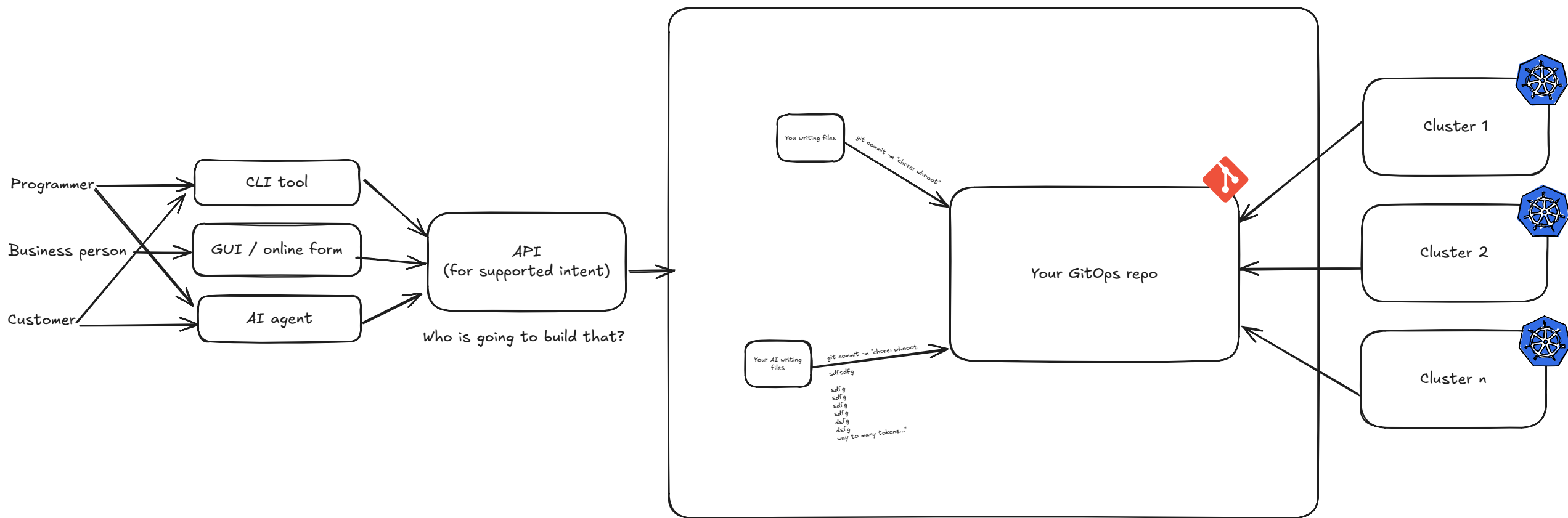
Then they do it!

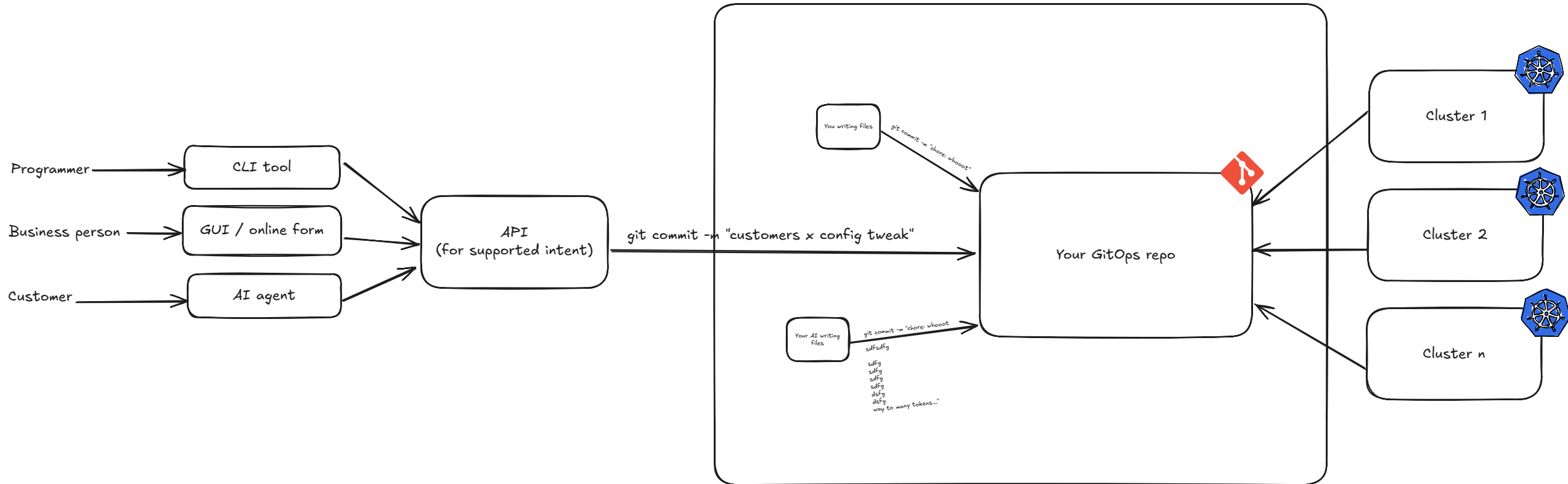
- layers (app-of-apps-of-apps-of-apps)
- custom CD pipelines
- complex helm templates
- referencing other sources of truth
 - open PRs in your repo with an ApplicationSet
 - specific helm charts in an online repo

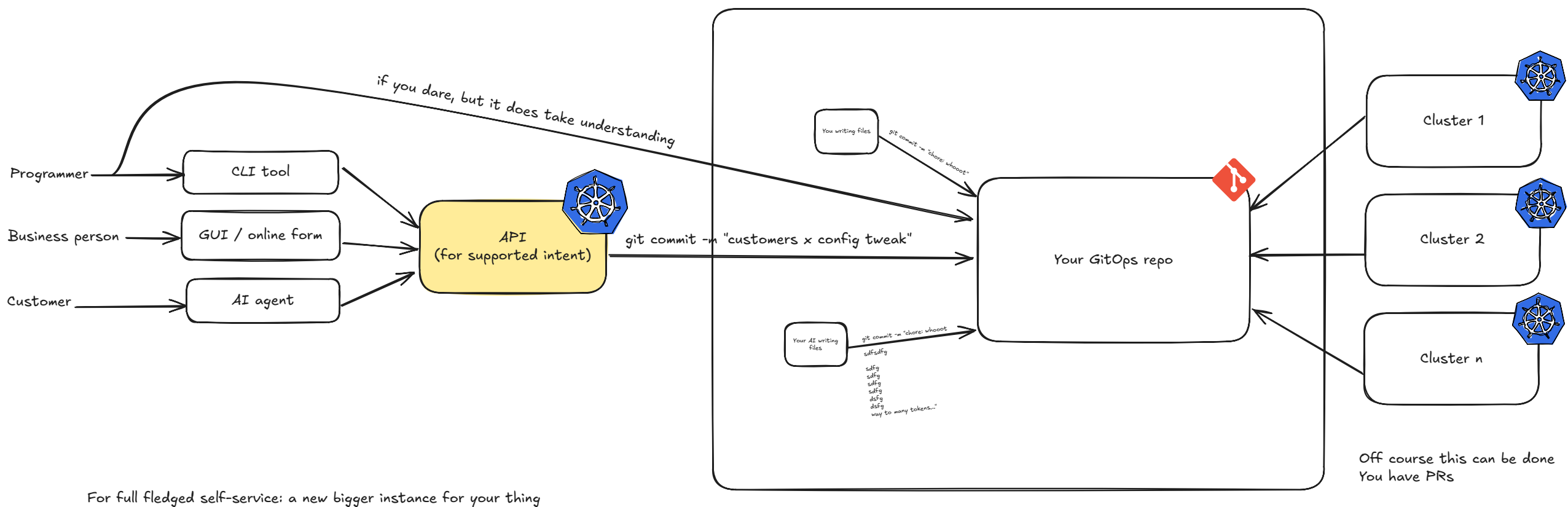
GitOps has a hard API

- Not everybody knows Git
- Do you expect customers to open up a PR?
- Conflicts are annoying
- Small typo in your file contents results in a failed deployment (can take a while!)









Dream mode

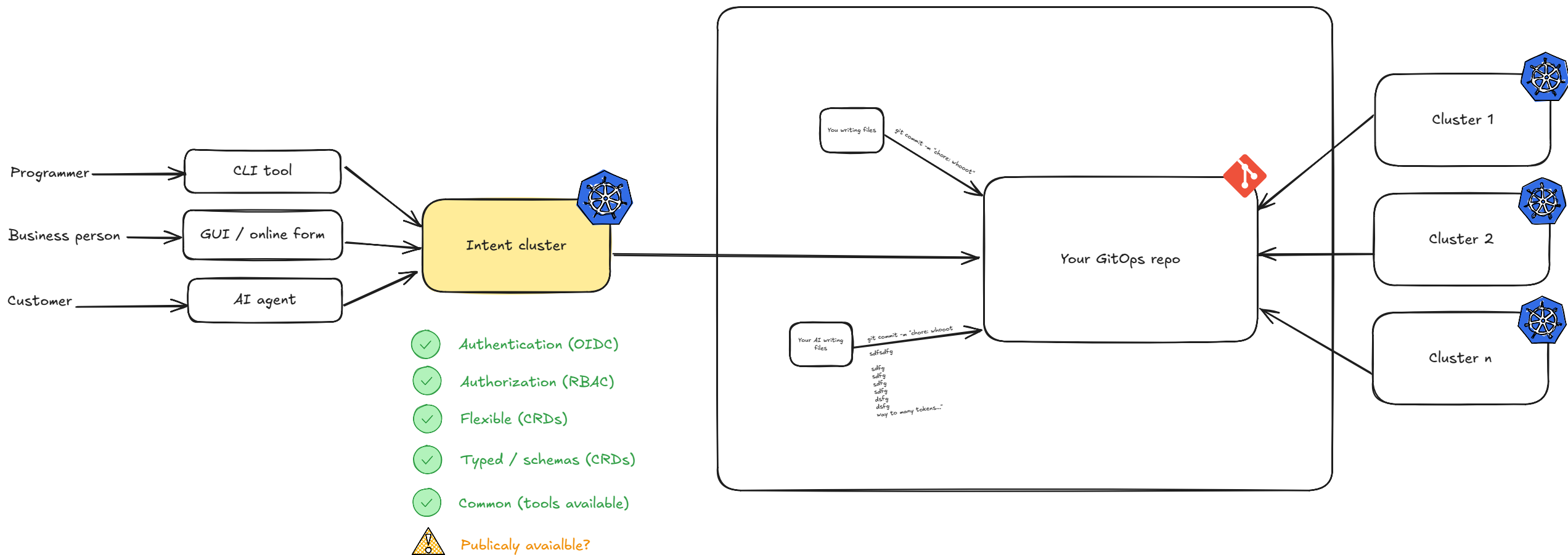
- For now: forget about complex structures
- Just keep a flat folder
- "intent" in plain manifests

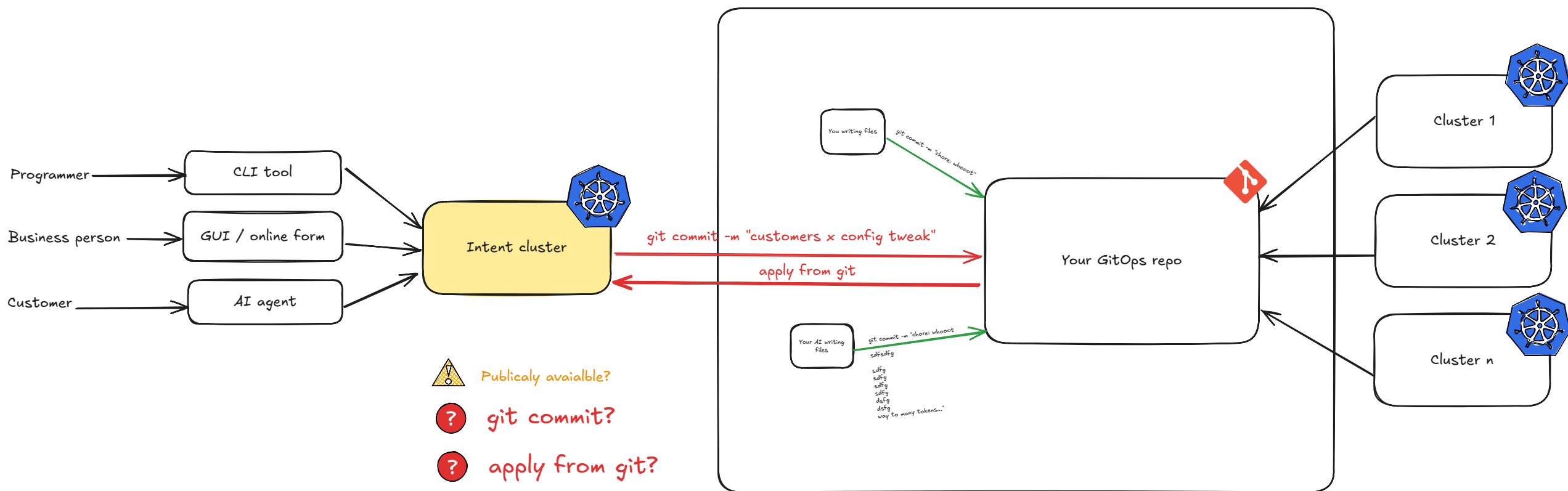
Dream mode (2)

- Keynote: KRO, Kratix or Crossplane are your friends!
- You can create and delete helm installs (ArgoCD application or HelmDeployment): but you don't edit the actual helm charts (that's a bad dream)
- Little bit of Kustomize is ok-ish, but be careful

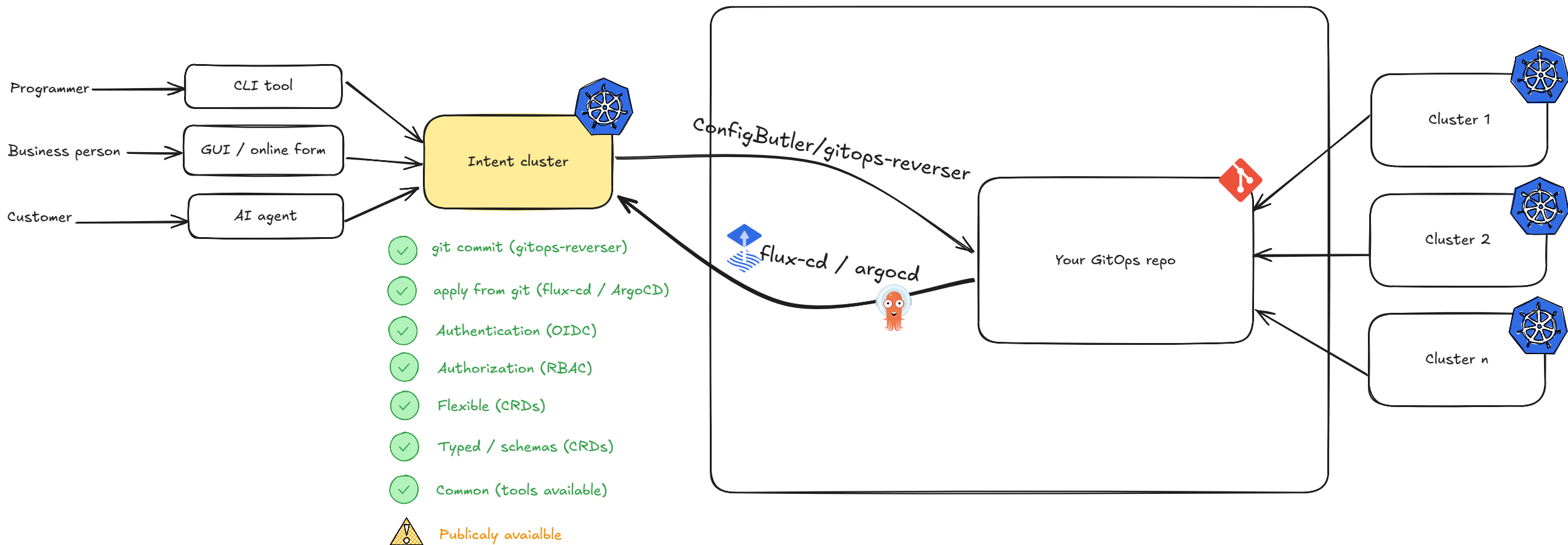
The revelation

The Kubernetes API was designed so that every API resource has a standard, serializable representation that can be stored in a file and submitted back to the API.





Ehhh wait: kube-api-server doesnt know anything about Git normally.



N clusters, and you want to "guard" changesets

Don't forget that n clusters can watch your GitOps folder.

Most changes should also first happen on a branch, so that you can create your PR.

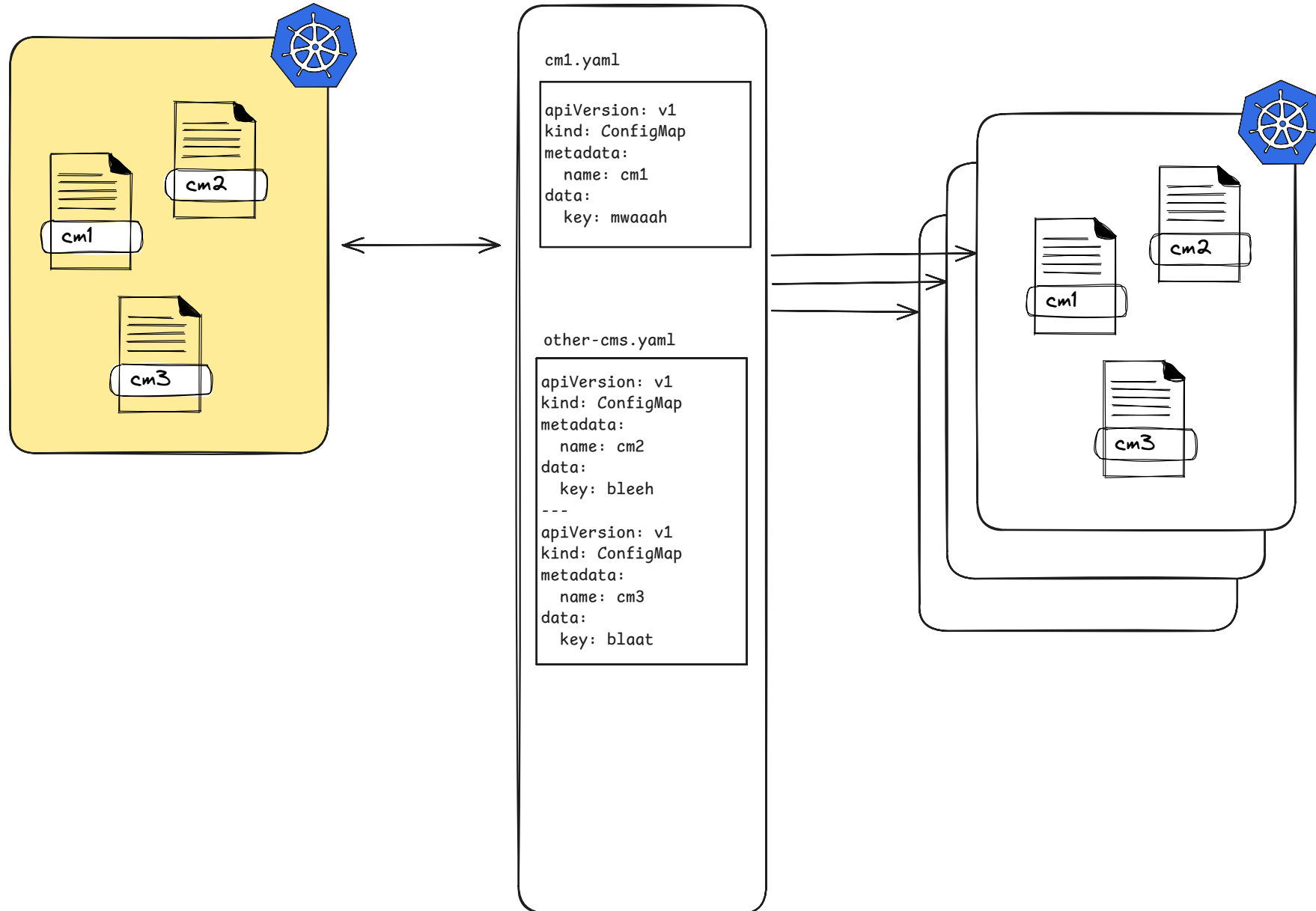
You should have the freedom to delete and create as you want.

But it should only be released on merge to main

Create a single data-only cluster per branch

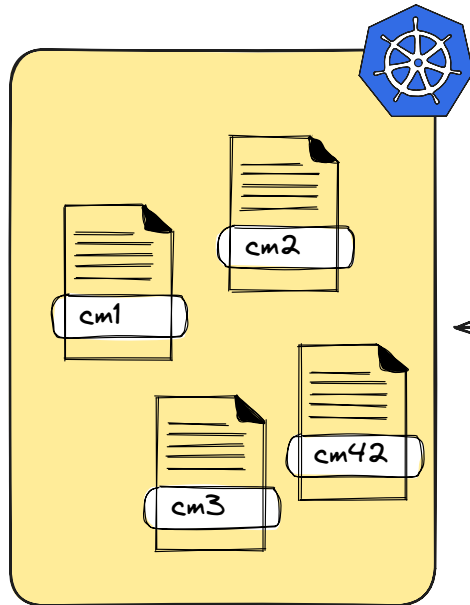
- Load all existing manifests as a starting point (t=0)
- Make your change in Git
 - ArgoCD syncs to your data-only cluster
- Make a change in your intent objects
 - gitops-reverser converts your changes into commits
- Remove your data-only cluster at merge

Intent cluster
API First "editor"
(data-only)

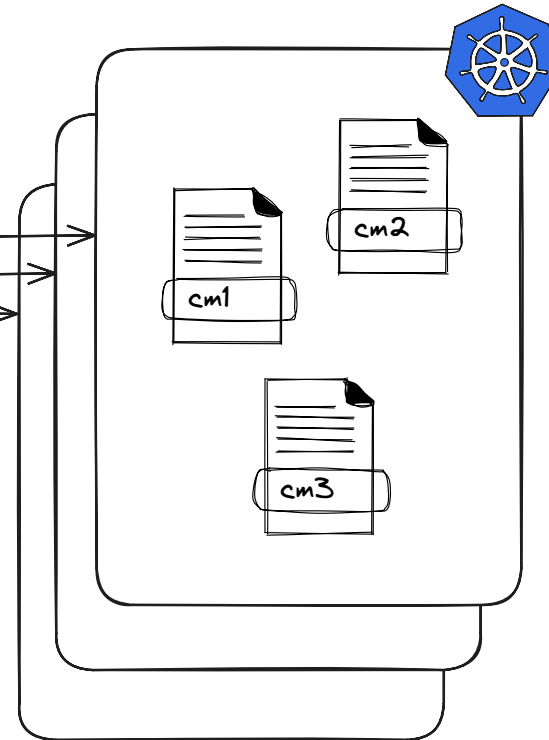
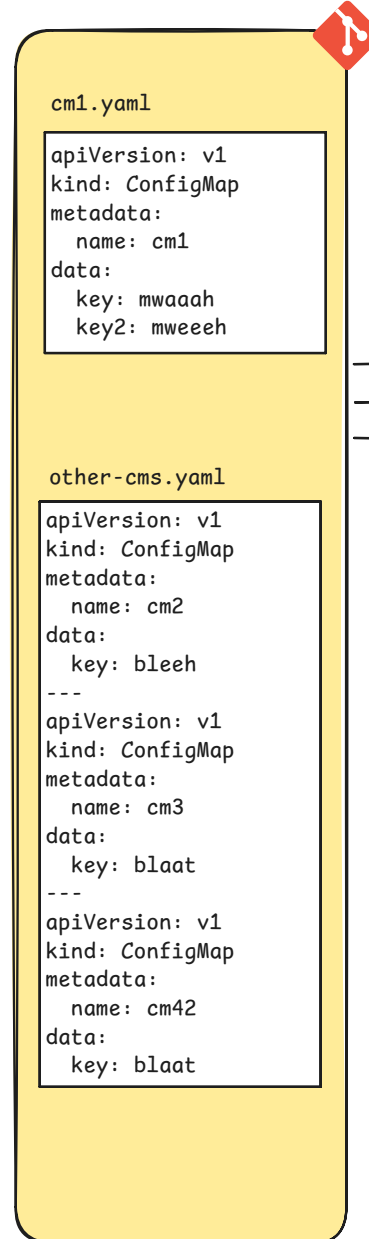


Intent cluster

API First "editor"
(data-only)

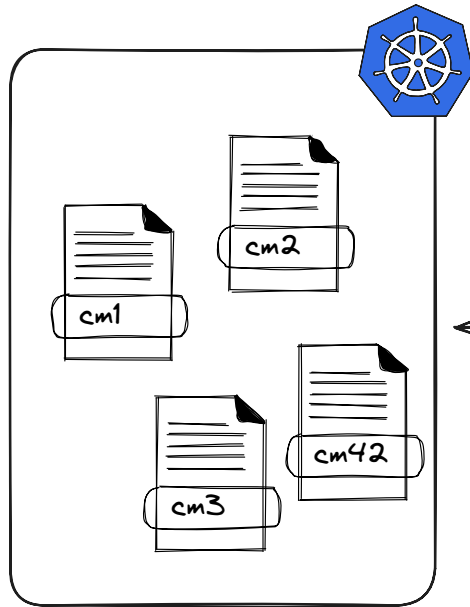


branch: feature1

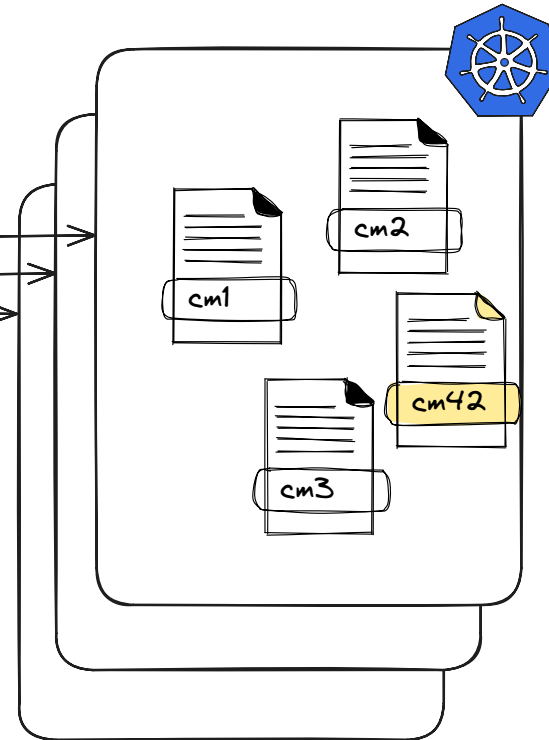
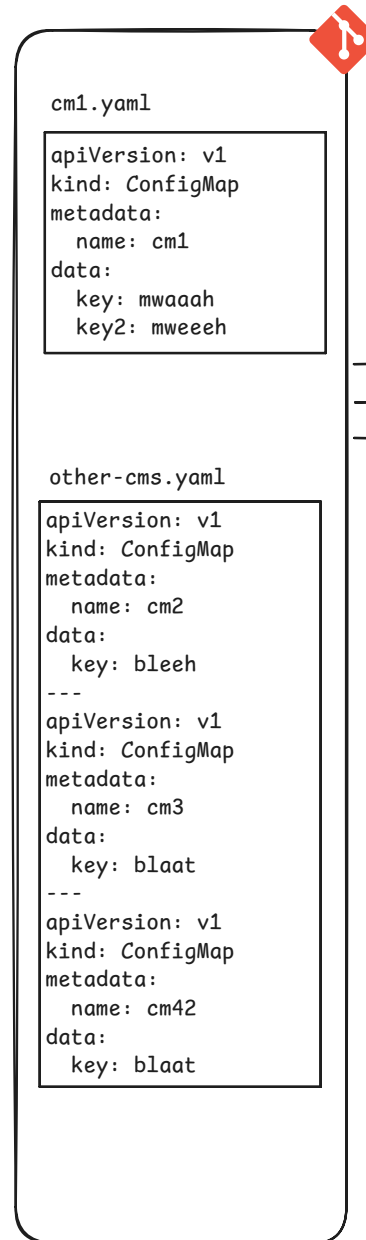


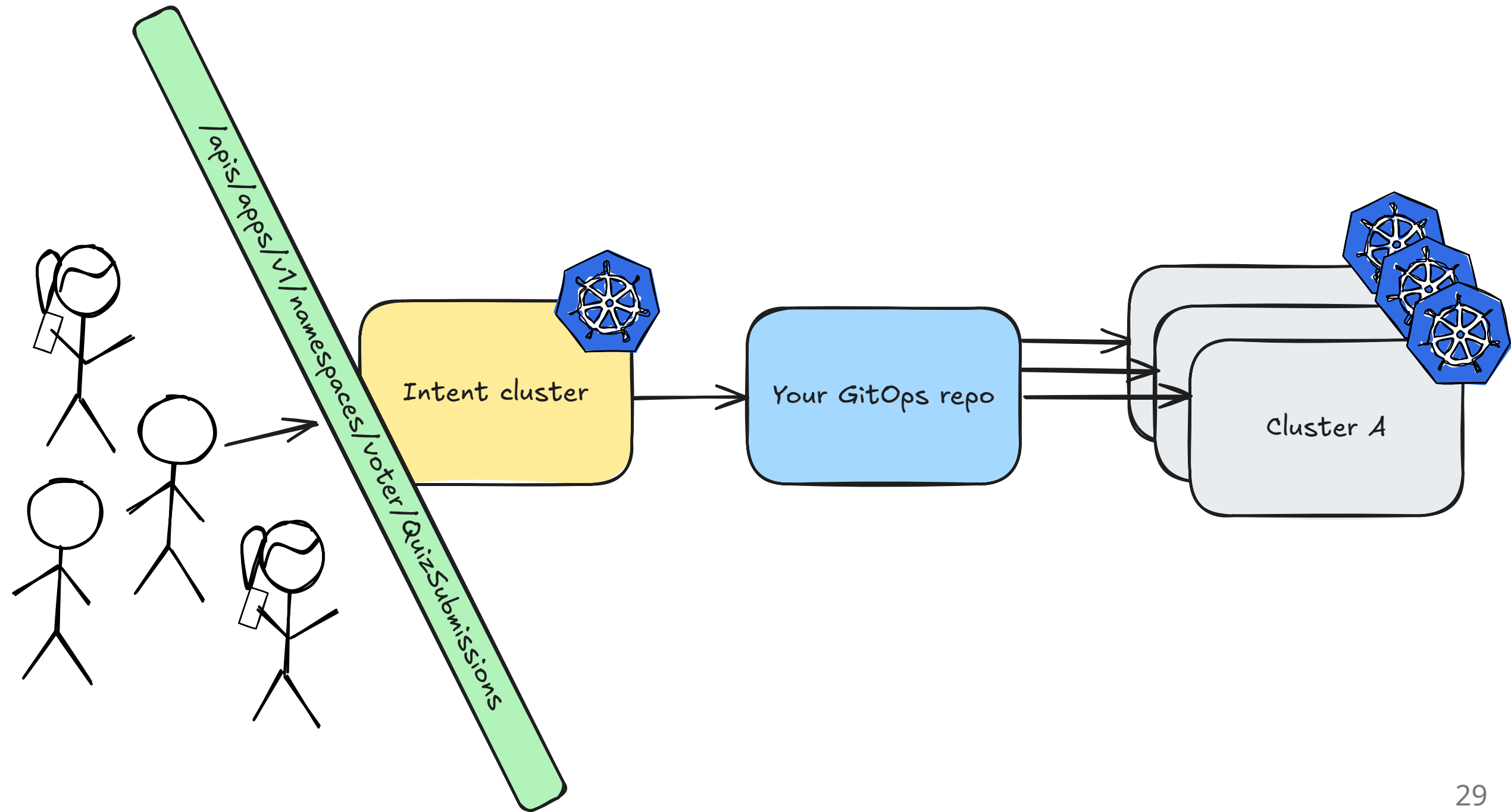
Intent cluster

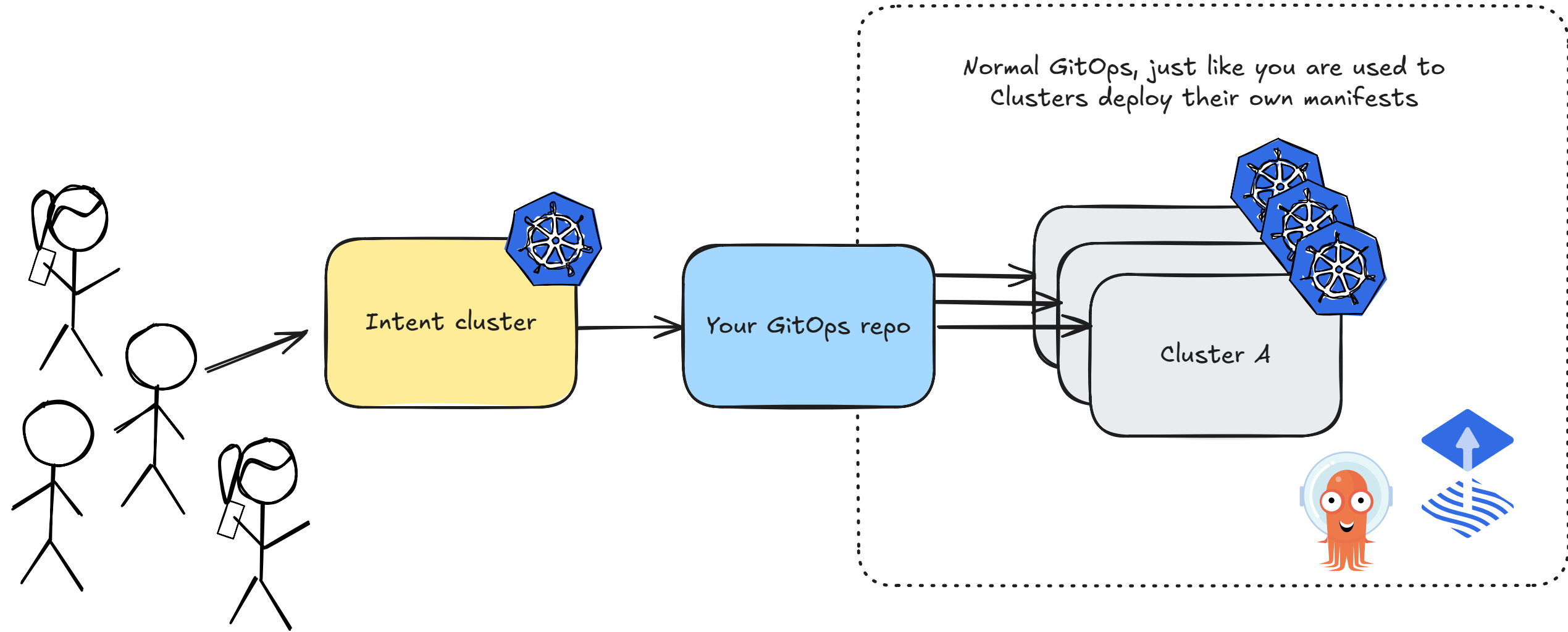
API First "editor"
(data-only)

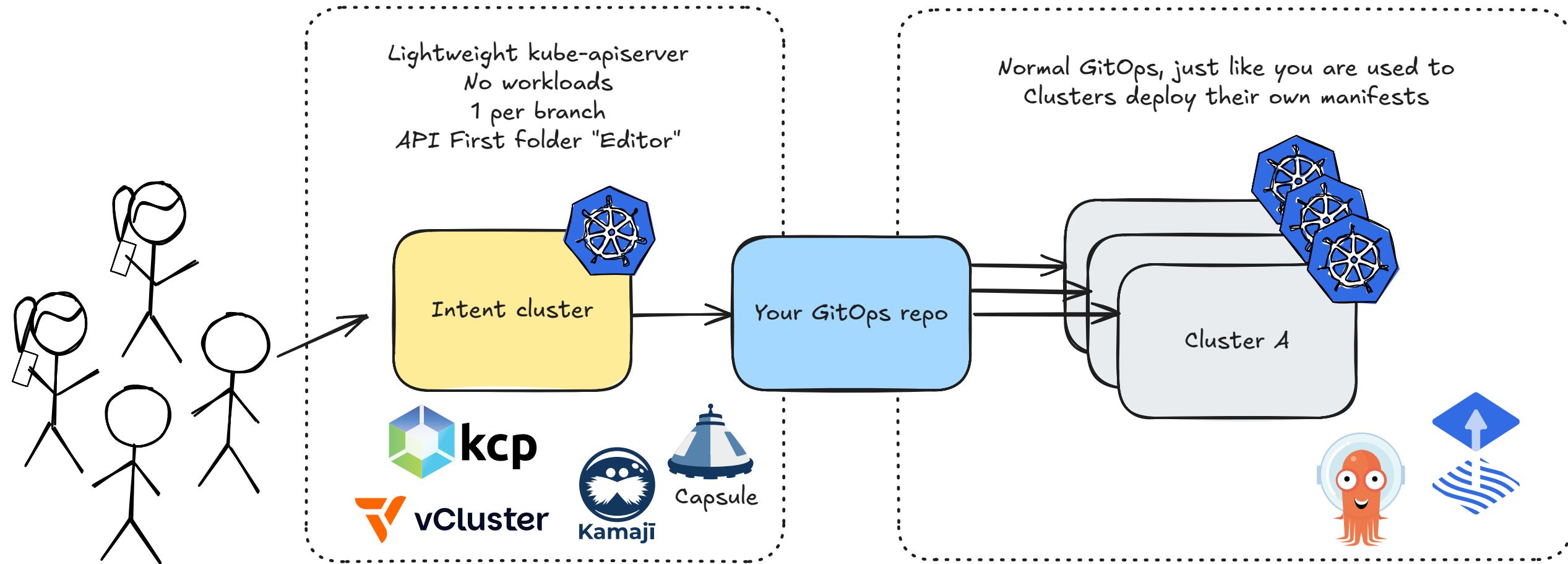


branch: main









Demo 1

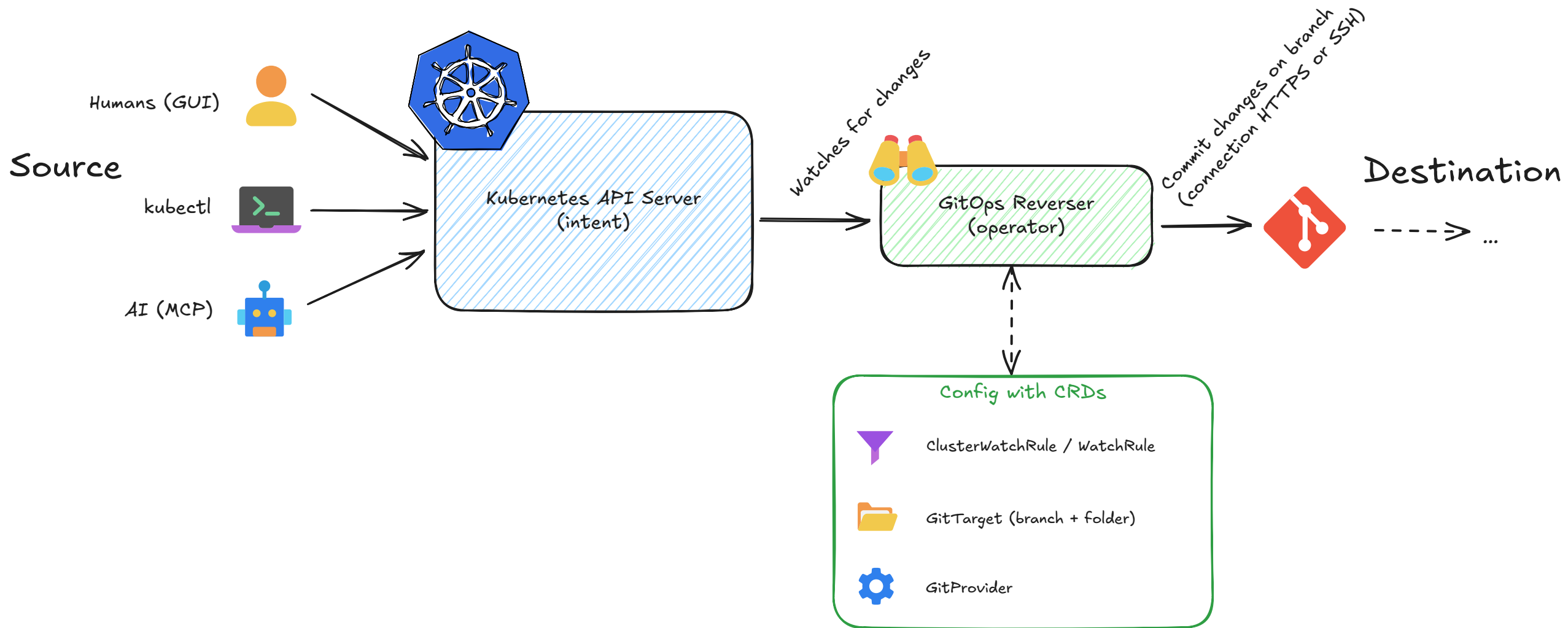
Who you are, who decides what you may do, and what a vote actually is

Scan. Pick a nickname you are happy for a room full of strangers to read.

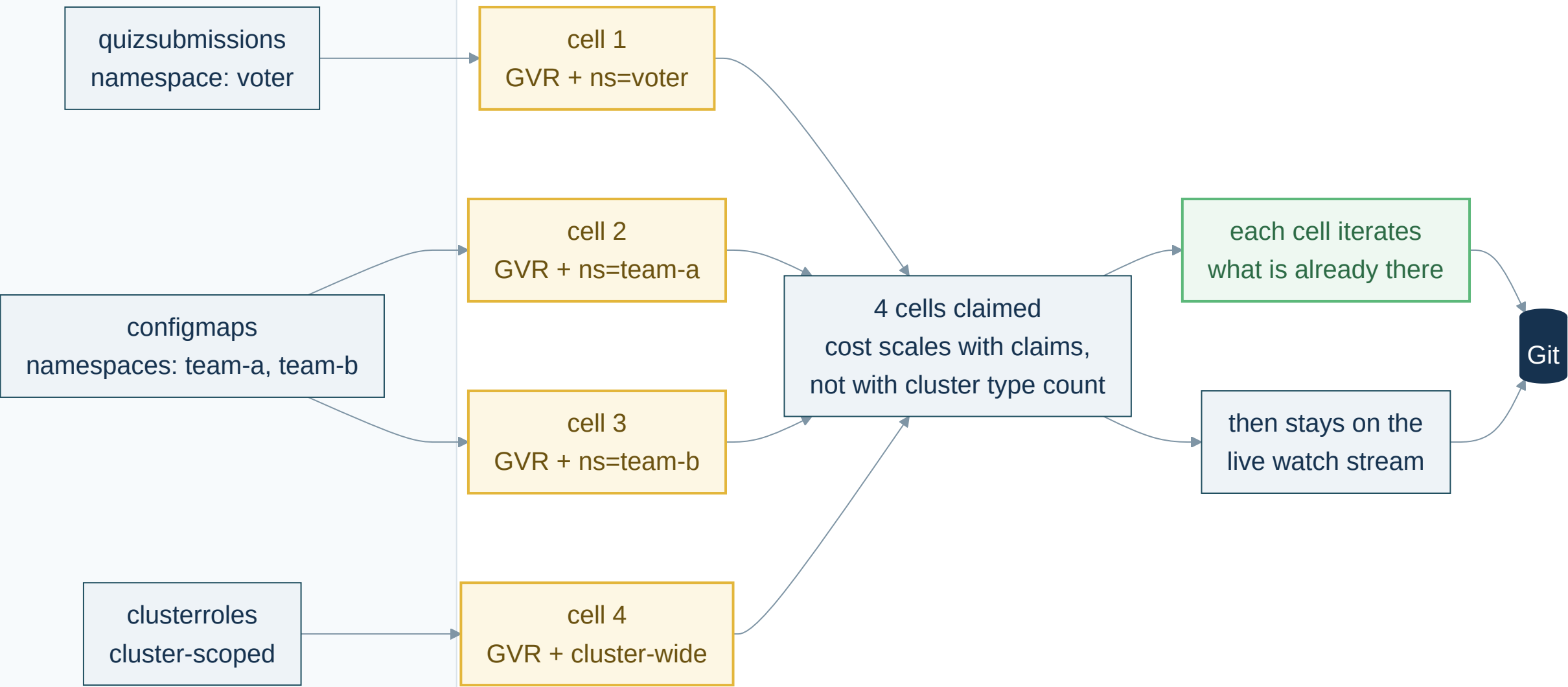
How do you observe kube-apiserver?

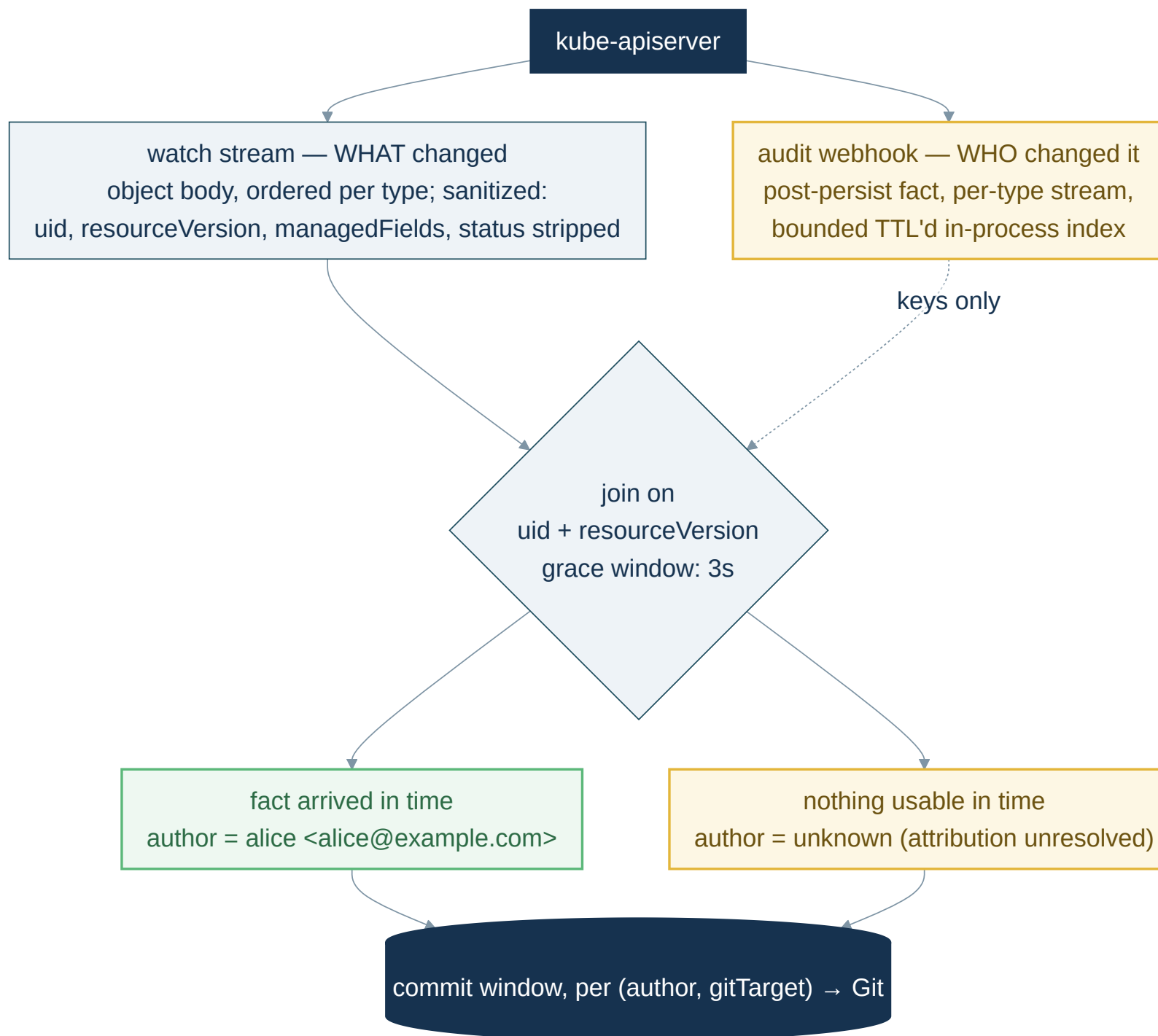
Mechanism	Tells you <i>who</i>	Fires	AKS/EKS/...
Watch stream	No	after commit	Yes
Audit webhook	Yes	after commit	No
Mutating webhook	Yes	<i>before</i> commit	Yes
Validating webhook	Yes	<i>before</i> commit	Yes
Audit file	Yes	after commit	No

The watch says **what** changed. The audit webhook says **who** changed it. They "merge" on `uid + resourceVersion`.



What a WatchRule / ClusterWatchRule claims





So why again?

- Leave your production clusters alone
- The data-only cluster can sit in a different network (also more acceptable on the public internet)
- Of course you can only enter with your own identity (OIDC)

Won't that create an infinite loop?

- gitops-reverser, ArgoCD and FluxCD stop when the spec is as desired
- No looping
- gitops-reverser controls when the sync is done

So you have commits

Who 

When 

What 

Why  -> gitops-reverser CommitRequest

Commit author and comitter

```
> git show --format=fuller <sha>
Author:      Simon7 <simon7@koudijs.dev.test>
AuthorDate:  Wed Sep 16 03:51:25 2026 +0000
Commit:      ConfigButler Bot <bot@configbutler.ai>
CommitDate:  Wed Sep 16 03:51:25 2026 +0000

    chore(demo1): 1 change from demo:CgZzaW1vbjcSCXJvb20tcGFzcw

    - [CREATE] quizsubmissions/demo1-simon7
```

The author is the human. The committer is the bot.

Demo 2

The same actions, in Git

You are still signed in. You do not have to join again.

Next try this organisation-wide



"The _____ team needs a _____ database for their new _____ service.

Requirements: _____ tier, owned by team _____, cost centre CC-finance-__. A _____-sized instance is fine. Provision it now."



Everything
AI might
say or do

Good fit

- Full GitOps love: Who, When, What and Why
- Identify clear CRD-based, high-level resources that reflect intent

Don't

- Try to reverse advanced GitOps stuff (you will fail)
- Use it for your really transactional data

In short

- gitops-reverser reconciles API resources to Git
- Combine it with ArgoCD or FluxCD to sync from Git to API
- Branching requires a fresh data-only environment (or namespace*)

It's just rsync on steroids

Other options

- You could have something that quickly creates OCI artifacts from a commit on main
- Automatic "push" of someone click-ops-ing in ArgoCD's GUI
- What if every configuration API in the world used KRM?

Reverse GitOps

koudijs.dev

ConfigButler: a managed c

reversegitops.dev

Talks

Implementations

FAQ

ReverseGitOps

Reverse GitOps

API-first. Git-recorded. GitOps deploys.

[Read the manifesto ↓](#)[★ Star on GitHub 13](#)

 I've spoken at CozySummit Virtual 2026 · [View talks](#)

Also spoke at: KubeCon + CloudNativeCon Europe 2026 · Platform Engineering Day

An API-first operating model where users and agents submit validated intent through a typed API, while that intent is automatically recorded in Git as clean declarative YAML manifests for audit, promotion, and distribution.

PERCEIVED PROBLEM

GitOps gives us reproducibility, auditability, and a shared language for intent. One of the best ideas the infrastructure community has produced.

But it carries an assumption that is starting to strain: that every change should enter the system through Git.

Next

- reversegitops.dev
 - Slides at: <https://github.com/reverse-gitops/talks>
- Useful in any way? Fill a few questions on how I did!
- Find me on LinkedIn: Simon Koudijs

Questions?